Présentation Technique

26 mars 2026

Sommaire

Qu'est-ce que Hadoop ?

Apache Hadoop est un framework open-source pour le **stockage distribué** et le **traitement massif** de données.

- Créé par **Doug Cutting** (2006, Yahoo !)
- Inspiré des publications Google (GFS + MapReduce)
- Conçu pour fonctionner sur du **matériel standard** (commodity hardware)
- Gère des volumes de **pétaoctets**

Hadoop

2006 – aujourd'hui
Version 3.x
Licence Apache 2.0
Écrit en Java

Pourquoi Hadoop ?

Le problème

Les données croissent exponentiellement : logs, capteurs IoT, réseaux sociaux, transactions...
Un seul serveur ne peut plus tout stocker ni tout traiter.

La solution Hadoop

- **Scale-out** : ajouter des machines plutôt qu'en acheter une plus grosse
- **Tolérance aux pannes** : les données sont répliquées automatiquement
- **Coût réduit** : fonctionne sur du matériel standard
- **Traitement local** : le calcul va vers les données (pas l'inverse)

Les 3 piliers de Hadoop

Où sont les données ?

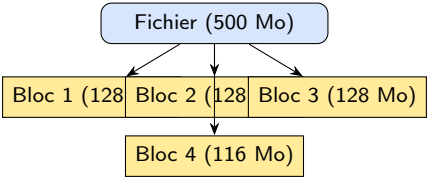
Qui gère les tâches ?

Comment traiter ?

HDFS – Principes

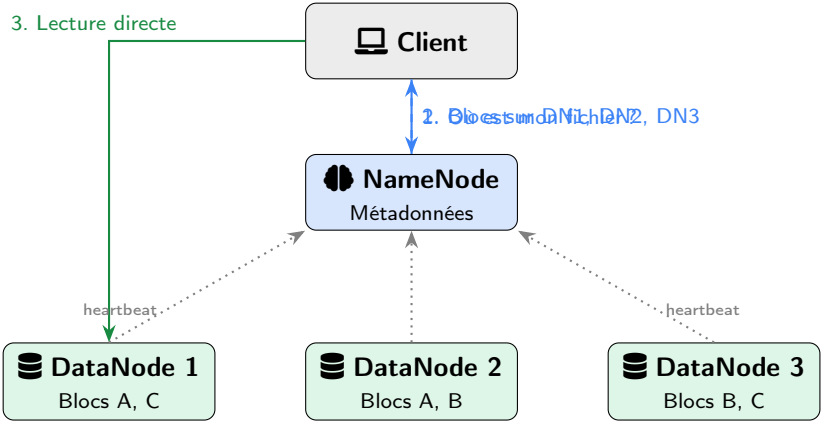
HDFS = système de fichiers distribué.

- Les fichiers sont découpés en **blocs** (128 Mo par défaut)
- Chaque bloc est **répliqué 3 fois** sur des machines différentes
- Optimisé pour la **lecture séquentielle** de gros fichiers
- Modèle **write-once, read-many**



Chaque bloc est répliqué
3 fois sur des nœuds différents

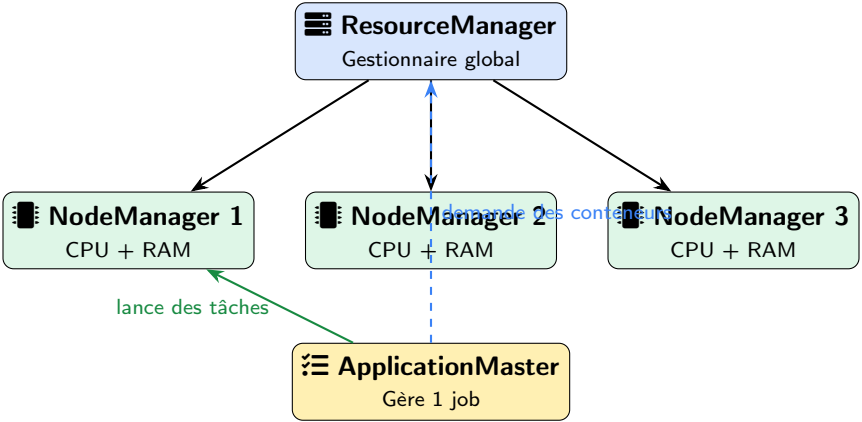
HDFS – Architecture NameNode / DataNode



Rôles dans HDFS

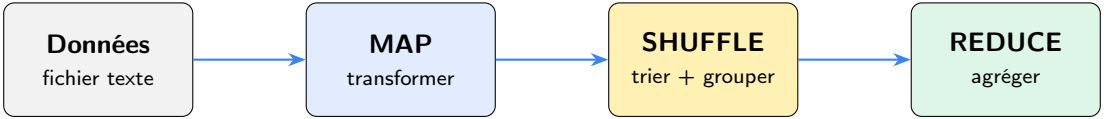
Composant	Rôle
NameNode	Serveur maître. Stocke les **métadonnées** : arborescence des fichiers, emplacement des blocs, permissions. Ne stocke **aucune donnée** réelle.
DataNode	Serveur esclave. Stocke les **blocs de données** sur le disque local. Envoie un *heartbeat* au NameNode toutes les 3 secondes.
Client	Lit/écrit des fichiers via le NameNode (métadonnées) puis communique directement avec les DataNodes (données).

YARN – Gestion des ressources



ResourceManager : alloue les ressources du cluster.
NodeManager : gère les ressources d'un nœud.
ApplicationMaster : coordonne l'exécution d'un job.

MapReduce – Principe




```
ligne 1
ligne 2
ligne 3
...
```

```
clé1 → 1
clé2 → 1
clé1 → 1
...
```

```
clé1 → [1, 1]
clé2 → [1]
...
```

```
clé1 → 2
clé2 → 1
...
```

MapReduce – Les 3 phases en détail

Phase MAP

- Lit les données **ligne par ligne**
- Émet des paires `clé → valeur`
- S'exécute en **parallèle** sur chaque nœud
- Pas de communication entre mappers

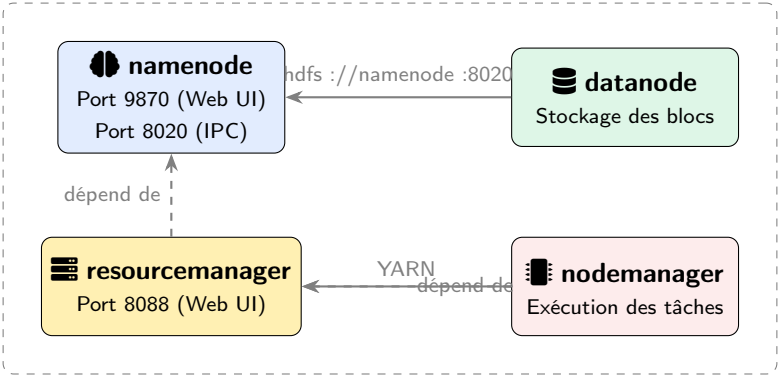
Phase SHUFFLE

- **Automatique** (faite par Hadoop)
- Trie les paires par clé
- Regroupe les valeurs par clé
- Transfère les données entre nœuds

Phase REDUCE

- Reçoit `clé → [val1, val2, ...]`
- **Agrège** les valeurs (somme, max, moyenne...)
- Écrit le résultat final dans HDFS

Notre cluster – docker-compose



docker-compose.yml

```yaml
services:
  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    environment:
      - CLUSTER_NAME=test
      - CORE_CONF_fs_defaultFS=hdfs://namenode:8020
    ports:
      - "9870:9870"       # Interface web HDFS

  datanode:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    environment:
      - CORE_CONF_fs_defaultFS=hdfs://namenode:8020

  resourcemanager:
    image: bde2020/hadoop-resourcemanager:2.0.0-hadoop3.2.1-java8
    ports:
      - "8088:8088"       # Interface web YARN

  nodemanager:
    image: bde2020/hadoop-nodemanager:2.0.0-hadoop3.2.1-java8
```

>_ Lancer : `docker compose up -d` | Arrêter : `docker compose down`

Cas pratique – Objectif

Scénario

Un serveur web génère des milliers de lignes de logs d'accès.
On veut analyser ces logs pour extraire des statistiques.

Questions auxquelles répondre :

1. Combien de requêtes par **code HTTP** ? (200, 404, 500...)
2. Quelles sont les **pages les plus visitées** ?
3. Quelle est la répartition par **méthode HTTP** ? (GET, POST...)
4. Comment le **trafic évolue-t-il par heure** ?

Outils utilisés

HDFS pour stocker les logs + **MapReduce** (Hadoop Streaming + awk) pour les analyser

Format des données

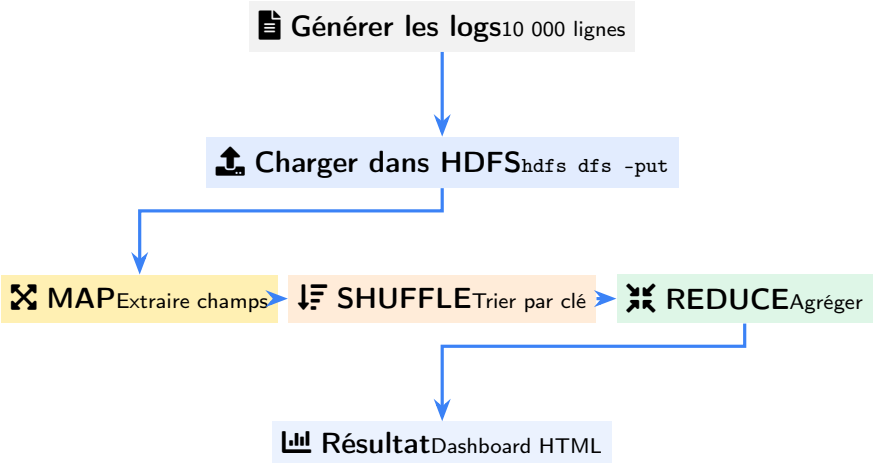
Chaque ligne de log suit le format **Apache Combined Log** :

```
192.168.1.10 - - [25/Mar/2026:10:00:01] "GET /index.html HTTP/1.1" 200 1024
192.168.1.12 - - [25/Mar/2026:10:00:03] "GET /missing.html HTTP/1.1" 404 512
192.168.1.15 - - [25/Mar/2026:10:00:07] "GET /api/data HTTP/1.1" 500 64
```

Décomposition des champs (séparés par des espaces) :

Champ	Nom	Exemple	Usage
$1	Adresse IP	192.168.1.10	–
$4	Date/Heure	[25/Mar/2026 :10 :00 :01]	Trafic par heure
$5	Méthode	"GET	Répartition GET/POST
$6	Page	/index.html	Pages visitées
$8	Code HTTP	200	Codes de réponse
$9	Taille	1024	Volume transféré

Mécanisme – Pipeline de traitement



Le Mapper (awk)

Le mapper lit chaque ligne et émet des paires `clé<TAB>valeur` :

```
BEGIN { OFS = "\t" }
{
    code = $8                # Code HTTP (200, 404, 500...)
    if (code ~ /^[0-9]+$/) {
        print code, 1        # Emet : 200<TAB>1
    }
}
```

Entrée (1 ligne de log) :

```
192.168.1.10 - - [25/Mar/2026:10:00:01] "GET /index.html HTTP/1.1" 200 1024
```

Sortie :

```
200     1
```

Le Reducer (awk)

Le reducer reçoit les paires **triées par clé** et additionne les compteurs :

```
BEGIN { FS = "\t"; OFS = "\t" }
{
    if ($1 == prev) {
        count += $2        # Meme cle : accumuler
    } else {
        if (prev != "") print prev, count
        prev = $1          # Nouvelle cle
        count = $2
    }
}
END { if (prev != "") print prev, count }
```

Entrée (triée par Hadoop) :

```
200     1
200     1
200     1
404     1
404     1
```

Lancement du job

```
hadoop jar hadoop-streaming-3.2.1.jar \
    -D stream.num.map.output.key.fields=2 \
    -files /tmp/mapper.awk,/tmp/reducer.awk \
    -mapper  "awk -f /tmp/mapper.awk" \
    -reducer "awk -f /tmp/reducer.awk" \
    -input  /user/root/input/access.log \
    -output /user/root/output/results
```

Ce que fait Hadoop :

1. Découpe le fichier d'entrée en **splits**
2. Lance le **mapper** sur chaque split (en parallèle sur les DataNodes)
3. **Trie et regroupe** les paires clé-valeur (shuffle)
4. Lance le **reducer** pour agréger les résultats
5. Écrit le résultat dans HDFS (`part-00000`)

Résultats – Codes HTTP

Code	Requêtes
200 OK	5 846
301 Redirect	812
403 Forbidden	841
404 Not Found	809
500 Server Error	835
502 Bad Gateway	857
Total	**10 000**

Interprétation :

- ~58% de succès (200)
- ~17% d'erreurs serveur (5xx)
- ~8% de pages non trouvées (404)
- ~8% d'accès refusés (403)

Action

Les erreurs 500/502 sont élevées → investiguer les endpoints `/api/crash` et `/api/timeout`.

Résultats – Pages et méthodes

Top 5 des pages les plus visitées :

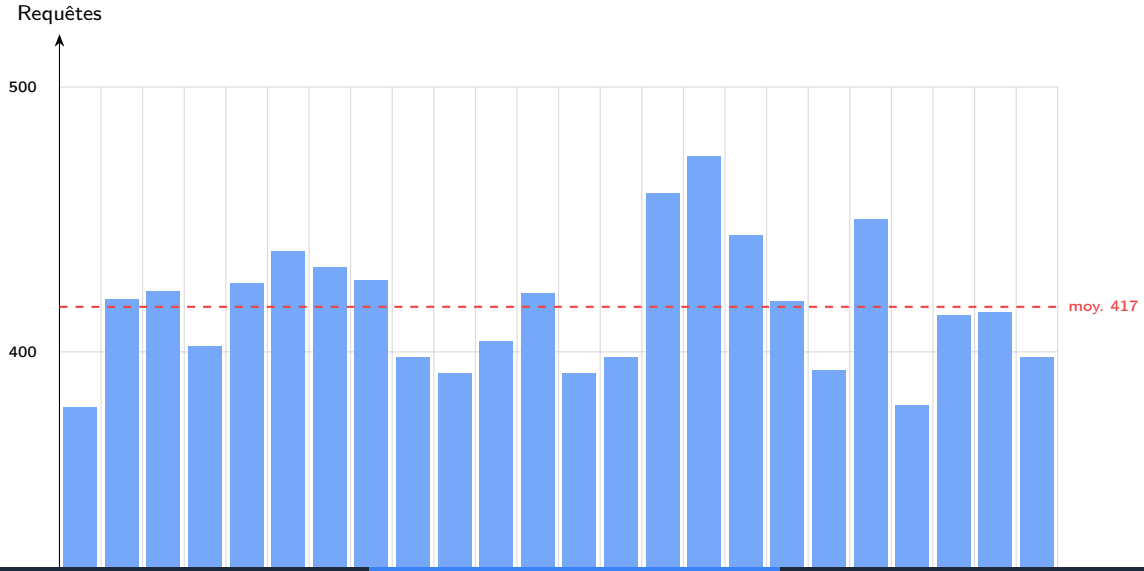
Page	Hits
/login	439
/deprecated	431
/search	423
/api/timeout	418
/blog/post-2	416

Répartition par méthode HTTP :

Méthode	Requêtes
GET	5 757 (57,6%)
POST	1 438 (14,4%)
PUT	1 412 (14,1%)
DELETE	1 393 (13,9%)

ℹ Le trafic est à majorité GET (lecture).

Résultats – Trafic par heure



Interfaces de monitoring

HDFS – `localhost:9870`

- État du cluster
- Capacité et utilisation
- Datanodes actifs
- Navigateur de fichiers HDFS

YARN – `localhost:8088`

- Liste des applications
- Statut des jobs (SUCCEEDED, FAILED)
- Nombre de mappers / reducers
- Temps d'exécution et logs

Dashboard généré – `localhost:8888`

Le script `run-cas-pratique.sh` génère automatiquement un dashboard HTML interactif avec des graphiques Chart.js : doughnut (codes), barres (heures, pages), pie (méthodes).

Ce qu'on retient

Hadoop permet de :

- Stocker des données massives de manière distribuée (**HDFS**)
- Traiter ces données en parallèle (**MapReduce**)
- Gérer les ressources d'un cluster (**YARN**)
- Tolérer les pannes automatiquement

Dans notre cas pratique :

- 10 000 logs analysés en quelques minutes
- 4 dimensions extraites (codes, pages, méthodes, heures)
- Pipeline reproductible : un seul script
- Résultats visualisables dans le navigateur

Commande unique : `bash run-cas-pratique.sh` → Données → HDFS → MapReduce

Merci !

Questions ?

- `github.com/Achaire-Zogo/hadoop`
- Dashboard : `http://localhost:8888/dashboard.html`
- `bash run-cas-pratique.sh`